

CÁTEDRA DE CIBERSEGURIDAD  
· UNIVERSIDAD DE MÁLAGA

# WatchGate

Sistema de *scoring* de riesgo para la revisión automatizada de *pull requests* en pipelines CI/CD

Javier Martín Jurado · Pablo Jiménez Castro · Pablo Ayllón García  
Málaga, julio de 2026



UNIVERSIDAD  
DE MÁLAGA

| uma.es

# ÍNDICE

---

01 El problema

02 Estado del arte

03 Arquitectura

04 El contrato entre capas

05 Las cuatro capas

06 Formato de salida

07 Capa estática

08 Capa de dependencias

09 Capa de reputación

10 Capa semántica

11 Infraestructura necesaria

# 01 · EL PROBLEMA

---

La revisión humana, como única barrera, tiene limitaciones estructurales

Fatiga del revisor, sesgo de confianza hacia colaboradores habituales, y la dificultad de detectar intención maliciosa cuando se disimula dentro de un cambio que parece legítimo. No es hipotético: son tres campañas reales de los últimos dos años.

2024 · CVE-2024-3094

## XZ Utils

Puerta trasera en liblzma, introducida por alguien que se ganó la confianza del proyecto durante años. El *payload* vivía en ficheros de test binarios y scripts de build, invisible en un diff normal.

2026 · AUR

## Atomic Arch

De varios cientos a más de 1.500 paquetes comprometidos en días. Los atacantes falsificaban nombre y correo de *commits* anteriores para simular continuidad con el mantenedor legítimo.

2026 · Wiz Research

## prt-scan

Seis cuentas generando PRs maliciosos a escala con IA, con el *payload* adaptado automáticamente al lenguaje de cada repositorio objetivo, contra GitHub Actions.

# 02 · ESTADO DEL ARTE

## Las herramientas existentes cubren una única capa del problema

Socket.dev y GuardDog miran la dependencia externa, no el código del PR. BewAlre (Datadog) razona sobre el diff con LLM, pero es interna y cerrada. Ninguna cruza identidad declarada contra historial real, ni documenta cómo gobierna su propia IA.

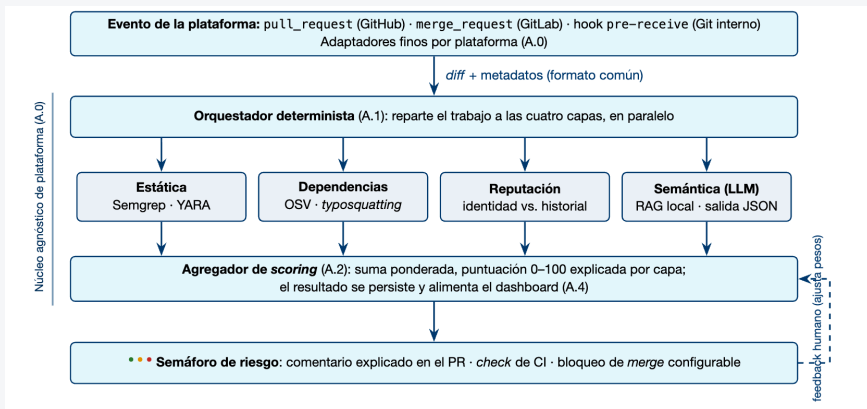
| Herramienta                         | Abierta        | Multiplat. | Identidad vs. hist. | Explicable     | Gobierna LLM   |
|-------------------------------------|----------------|------------|---------------------|----------------|----------------|
| Socket.dev / GuardDog               | ~ <sup>1</sup> | X          | X                   | ~ <sup>2</sup> | n/a            |
| BewAlre ( <i>Datadog, interno</i> ) | X              | X          | X                   | X <sup>3</sup> | X <sup>3</sup> |
| Phylum / Aikido / Snyk              | X              | X          | ~                   | n/a            | n/a            |
| <b>WatchGate</b>                    | ✓              | ✓          | ✓                   | ✓              | ✓              |

<sup>1</sup>GuardDog es abierto; Socket.dev, no.    <sup>2</sup>Solo cubre la capa de dependencias.    <sup>3</sup>No documentado públicamente.

# 03 · ARQUITECTURA

## Un núcleo agnóstico, cuatro capas en paralelo

El núcleo no sabe nada de GitHub, GitLab ni del proveedor de LLM: solo procesa un *diff* normalizado. Los adaptadores finos traducen cada plataforma a ese contrato común.



## 04 · EL CONTRATO ENTRE CAPAS

Interfaz única: mismo diff de entrada, mismo resultado de salida

Cualquier capa nueva implementa esta interfaz y se registra por decorador; el orquestador nunca importa una capa concreta por nombre.

```
class AnalysisLayer(ABC):
    name: str

    @abstractmethod
    def analyze(self, diff: NormalizedDiff,
               metadata: dict) -> LayerResult:
        """Nunca lanza excepción no
        controlada; en error, devuelve
        skipped=True."""

    LAYER_REGISTRY: dict[str, type[AnalysisLayer]] = {}

    def register_layer(cls):
        LAYER_REGISTRY[cls.name] = cls
        return cls
```

```
class LayerResult(BaseModel):
    layer_name: str
    risk_score: int # 0-100
    justification: str
    category: RiskCategory | None
    confidence: Confidence | None
    skipped: bool = False
    skip_reason: str | None
    tool_calls_made: int = 0
```

### Fórmula del agregador (A.2)

$$\text{score} = \frac{\sum_i w_i \cdot \text{risk\_score}_i}{\sum_i w_i}$$

suma solo sobre capas activas (no skipped, peso > 0);  
se renormaliza si alguna se desactiva.

# 05 · LAS CUATRO CAPAS

---

Cada capa es un módulo intercambiable

Misma entrada (diff + metadata), misma salida (puntuación 0–100 + justificación en texto). Se puede desactivar una capa por proyecto, o cambiar el proveedor de LLM, sin tocar el resto.

## 3a

### Estática

Semgrep + YARA

eval/exec no literal, curl | bash en scripts de build, escritura a rutas sensibles, blobs base64 de alta entropía.

## 3b

### Dependencias

OSV + distancia de edición

Paquetes nuevos, CVEs contra OSV, y *typosquatting* (nombres a 1–2 caracteres de un paquete legítimo).

## 3c

### Reputación

reglas explícitas, no ML

Cruza identidad declarada (nombre/email) contra el historial real de la cuenta. El vector directo contra Atomic Arch.

## 3d

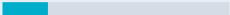
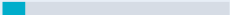
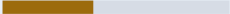
### Semántica

LLM + RAG + tools acotadas

Evalúa la intención del cambio, independientemente de quién parezca haberlo firmado.

## 06 · FORMATO DE SALIDA

### ■ WatchGate · Riesgo medio – 47/100

|                 |                                                                                   |        |
|-----------------|-----------------------------------------------------------------------------------|--------|
| Estática        |  | 20/100 |
| Dependencias    |  | 10/100 |
| Reputación      |  | 40/100 |
| Semántica (LLM) |  | 85/100 |

*"El cambio añade una llamada de red a un dominio externo no declarado en la documentación del proyecto, activada durante el proceso de build."*

→ **Revisión humana reforzada** antes de fusionar.

# 07 · CAPA ESTÁTICA

---

## Patrones de código peligrosos conocidos

Semgrep (reglas propias + registros públicos p/security-audit, p/secrets) sobre los hunks modificados, más YARA sobre el mismo contenido.

### Reglas Semgrep propias (5)

```
eval-exec-dynamic  
shell-true-subprocess  
curl-pipe-shell  
privilege-escalation-write  
network-call-in-build-script
```

### Reglas YARA (2)

```
long_base64_blob  
hex_encoded_shellcode_pattern  
  
entropía de Shannon > 4.5 bits/byte
```

### Regla de scoring

risk\_score = **máximo** de las severidades disparadas (nunca suma — evita inflar el score por volumen de hallazgos menores). Caso límite: diff sin ficheros de código → risk\_score = 0, sin invocar Semgrep.

# 08 · CAPA DE DEPENDENCIAS

---

## Paquetes nuevos, CVEs conocidas y *typosquatting*

Solo se activa si el diff toca un manifiesto: `package.json`, `requirements.txt`, `Pipfile`, `PKGBUILD`, `Cargo.toml`.

### Consulta OSV

POST `api.osv.dev/v1/query` por cada dependencia nueva o con versión cambiada; caché SQLite local con TTL de 24h.

### Typosquatting

distancia de Levenshtein  $\leq 2$  contra el top-N de paquetes por descargas de ese ecosistema (p. ej. "1odash" vs. "lodash").

### Scoring (máximo entre señales)

CVE crítica/alta  $\rightarrow 90$  · typosquatting probable  $\rightarrow 75$  · script de instalación con hallazgos de red/ejecución  $\rightarrow 80$ . Caso límite: fallo de red a OSV **no** sube el score, se anota como "no verificable".

# 09 · CAPA DE REPUTACIÓN

## Identidad declarada frente a historial real de la cuenta

Reglas explícitas, no ML. Nunca hace llamadas HTTP: toda la metadata llega ya resuelta desde el adaptador de plataforma.

| Señal                                                 | Puntos |
|-------------------------------------------------------|--------|
| Cuenta con menos de 30 días de antigüedad             | +30    |
| Cero contribuciones previas al repositorio            | +20    |
| Email del commit no verificado por la plataforma      | +25    |
| Repo firma habitualmente, este commit no está firmado | +30    |
| Firmado, pero con clave nunca vista para ese login    | +40    |

### Agregación y caso de referencia

Suma de las señales activas, tope en 100. Metadata simulando el caso Atomic Arch (cuenta de 2 días, 0 contribuciones, email no verificado)  $\rightarrow$  `risk_score`  $\geq$  75. Si no llega ReputationMetadata completo, la capa se omite (`skipped=True`), nunca inventa un score.

# 10 · CAPA SEMÁNTICA

---

La evaluación no se basa únicamente en el contenido visible del diff

Antes de preguntar, el sistema ya le sirve contexto de casos reales parecidos (RAG). Durante el análisis, el modelo puede pedir investigar más por su cuenta — hasta un límite duro.

## 01 Control de coste

¿Hay presupuesto de tokens este mes? ¿Este diff exacto ya se analizó? Si no hay presupuesto, se omite sin llamar al LLM.

## 02 RAG — recuperación automática

El propio sistema resume el diff y busca los 3 casos más parecidos en un corpus local (XZ Utils, Atomic Arch, prt-scan, MITRE ATT&CK). El LLM no lo pide: ya le llega servido.

## 03 El LLM investiga (*si lo necesita*)

En este paso el modelo decide si necesita más contexto: puede invocar hasta 3 herramientas antes de responder.

`lookup_package_registry` · `get_commit_history` · `fetch_referenced_file`

## 04 Salida estructurada

`risk_score` · `category` · `justification` · `confidence` — JSON validado, se cachea, se combina con las otras 3 capas.

# 11 · INFRAESTRUCTURA NECESARIA

---

## Tenemos acceso a Google Cloud Platform

No tenemos experiencia previa con GCP: esto es lo que creemos que vamos a necesitar, a falta de validarlo según avance el proyecto — no una arquitectura cerrada.

| Servicio                               | Para qué lo necesitamos                                                                                                                                                   |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cloud SQL (Postgres + pgvector)</b> | Sustituye el SQLite del dashboard y guarda los embeddings del corpus RAG en la misma base de datos (extensión pgvector), en vez de montar un servicio de vectores aparte. |
| <b>Cloud Run</b>                       | Backend del dashboard (FastAPI) sin gestionar servidores; escala a cero entre análisis.                                                                                   |
| <b>Secret Manager</b>                  | El backend en Cloud Run necesita el secreto OAuth de GitHub y la contraseña de Cloud SQL; viven aquí cifrados, no en variables de entorno en claro.                       |
| <b>Cloud Logging</b>                   | Ingesta del logging JSON estructurado que ya exige el núcleo (A.0).                                                                                                       |
| <b>Vertex AI (Gemini)</b>              | Proveedor de LLM alternativo a Anthropic para la capa semántica (LLMClient ya es sustituible).                                                                            |
| <b>Identity Platform</b>               | Que un cliente empresarial inicie sesión con su propio proveedor de identidad (Okta, Azure AD...) en vez de una cuenta de GitHub, sin programar cada federación a mano.   |